

Merge K sorted Lists

Problem Statement: Given heads of k sorted linked lists as an array called heads, merge them into one single sorted linked list and return the head of that list.

Examples

Input : heads = [[head -> 1 -> 2 -> 3 -> 4], [head -> -4 -> -3], [head -> -5 -> -3 -> 1 -> 2 -> 3 -> 4]]

Output: head -> -5 -> -4 -> -3 -> -3 -> 1 -> 1 -> 2 -> 2 -> 3 -> 3 -> 4 -> 4

Explanation: head -> -5 -> -4 -> -3 -> -3 -> 1 -> 1 -> 2 -> 2 -> 3 -> 3 -> 4 -> 4

The nodes in bold come from the 3rd list, the underlined nodes come from the 2nd list, rest are from the 1st list.

Input : heads = [[head -> -5 -> -4 -> -1], [head -> 10 -> 11 -> 12]]

Output : head -> -5 -> -4 -> -1 -> 10 -> 11 -> 12

Explanation : head -> -5 -> -4 -> -1 -> 10 -> 11 -> 12

The nodes in bold come from the 1st list, rest are from the 2nd list.

Brute Force Approach

Algorithm

To merge `k` sorted linked lists, the most basic thing we can do is just collect all the node values from every list, store them into one big container, sort it, and then rebuild a single sorted linked list out of those sorted values. This approach doesn't leverage the fact that the input lists are already sorted so it's inefficient but it's a good starting point to understand the flow of the problem.

It's like gathering all the exam papers, sorting by marks again manually, and then preparing the final rank list. No merging, just raw sorting. This helps build clarity before jumping into optimized approach.

- Initialize an empty array to store all node values from every list.
- Iterate through each linked list and push every node's value into the array.
- Sort the array of values.
- Create a new dummy head for the final linked list.
- Iterate over the sorted array and create a new node for each value, appending it to the final list.
- Return the node next to the dummy head as the actual head of the merged list.



list 1 : 1 → 4 → 5

list 2 : 1 → 3 → 4

list 3 : 2 → 6



dummy = listNode(0)

cur = dummy

insert

1

 → 1

insert

1

 → 1 → 1

insert

2

 → 1 → 1 → 2

insert

3

 → 1 → 1 → 2 → 3

insert

4

 → 1 → 1 → 2 → 3 → 4

insert

4

 → 1 → 1 → 2 → 3 → 4 → 4

insert

5

 → 1 → 1 → 2 → 3 → 4 → 4 → 5

insert

6

 → 1 → 1 → 2 → 3 → 4 → 4 → 5 → 6

Code

CppJavaPythonJavascript

```
import java.util.*;
// Definition for singly-linked list.
class ListNode {
    int val;
    ListNode next;

    // Constructor to initialize node with value
    ListNode(int val) {
        this.val = val;
        this.next = null;
    }
}

class Solution {
    // Function to merge k sorted linked lists using brute force
    public ListNode mergeKSortedLists(ListNode[] lists) {
        // List to collect all node values
        List<Integer> allValues = new ArrayList<>();

        // Traverse each linked list
        for (ListNode head : lists) {
            while (head != null) {
                allValues.add(head.val);
                head = head.next;
            }
        }

        // Sort the collected values
        Collections.sort(allValues);

        // Create dummy node for result list
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;

        // Create new nodes from sorted values
        for (int val : allValues) {
            curr.next = new ListNode(val);
            curr = curr.next;
        }

        return dummy.next;
    }
}

// Driver code
class Main {
    public static void main(String[] args) {
        // Example: [[1,4,5],[1,3,4],[2,6]]
        ListNode a = new ListNode(1);
        a.next = new ListNode(4);
        a.next.next = new ListNode(5);

        ListNode b = new ListNode(1);
        b.next = new ListNode(3);
        b.next.next = new ListNode(4);

        ListNode c = new ListNode(2);
        c.next = new ListNode(6);

        ListNode[] lists = new ListNode[]{a, b, c};
    }
}
```

```

        Solution sol = new Solution();
        ListNode result = sol.mergeKSortedLists(lists);

        while (result != null) {
            System.out.print(result.val + " ");
            result = result.next;
        }
    }
}

```

Complexity Analysis

Time Complexity: $O(N \log N)$, We traverse all the nodes once to collect their values, which takes $O(N)$ where N is the total number of nodes across all lists. Sorting these N values takes $O(N \log N)$. Building the new linked list from the sorted array takes $O(N)$. So the overall time complexity is $O(N \log N)$.

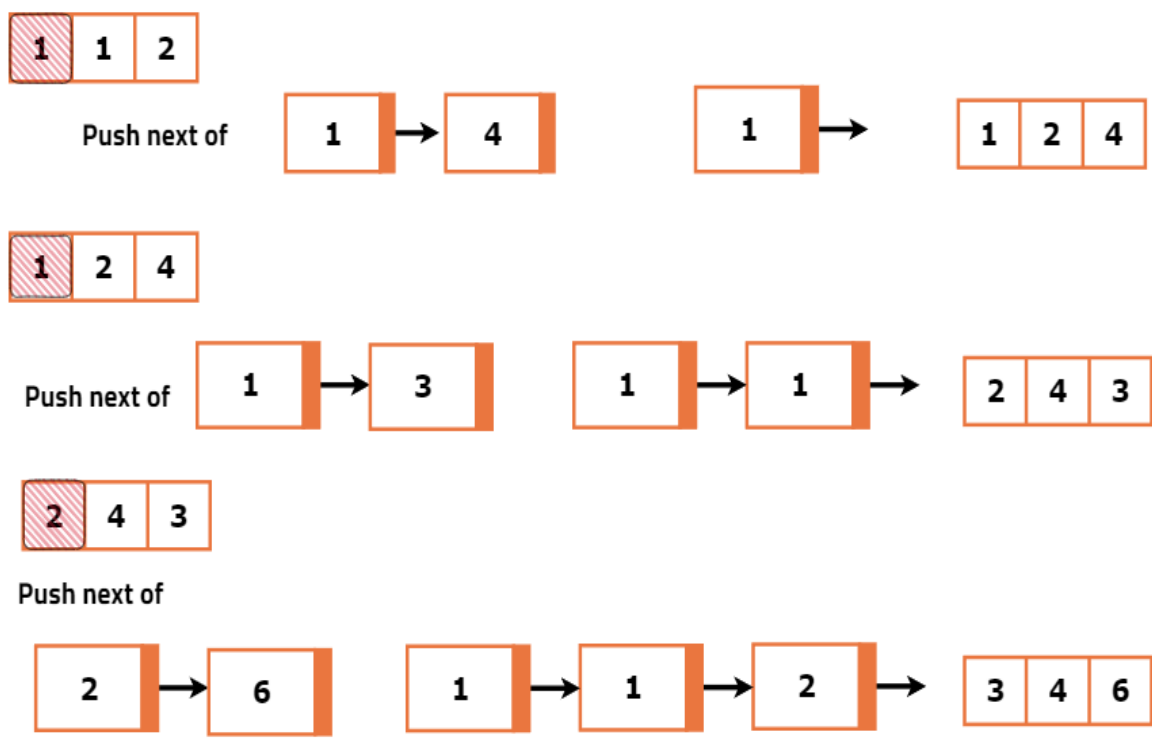
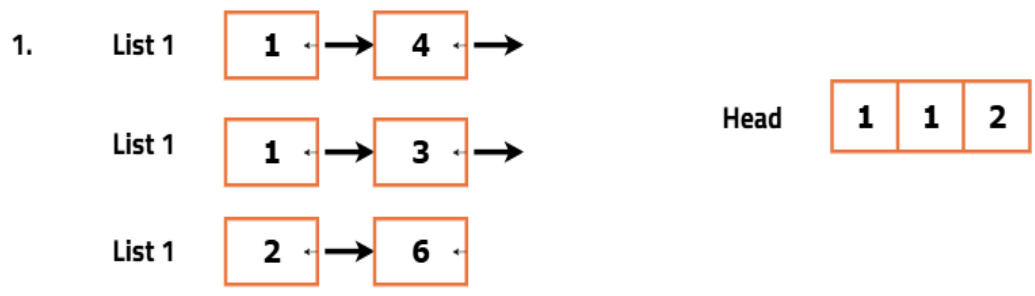
Space Complexity: $O(N)$, We use an additional array to store all the node values, which requires $O(N)$ space. Plus, the new merged linked list also takes $O(N)$ space, but since this is the required output, only the temporary storage is considered extra.

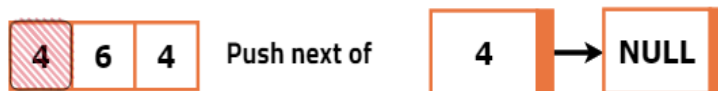
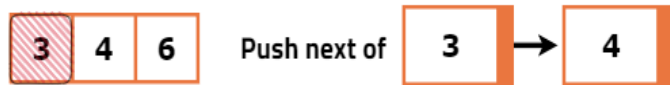
Optimal Approach

Algorithm

Instead of extracting all values and sorting them again, we can merge the lists by always picking the smallest current node among all lists. This can be achieved efficiently using a min-heap (or priority queue), where we store the current node of each list and always extract the node with the smallest value. By pushing the next node of the extracted node back into the heap, we ensure that at any point, the heap contains the next smallest potential candidates. This approach avoids unnecessary sorting and leverages the fact that each list is already sorted.

- Create a min-heap that stores the current head node of each list.
- Push the head of each non-empty list into the min-heap.
- Initialize a dummy node to build the resulting merged list.
- While the heap is not empty:
 - Pop the node with the smallest value from the heap.
 - Append this node to the result list.
 - If the popped node has a next node, push it into the heap.
- Return the node next to the dummy as the head of the merged list.





Code

CppJavaPythonJavascript

```
import java.util.*;

// Definition for singly-linked list
class ListNode {
    int val;
    ListNode next;
    ListNode(int x) {
        val = x;
        next = null;
    }
}

class Solution {
    // Function to merge k sorted linked lists using a min-heap
    public ListNode mergeKLists(ListNode[] lists) {
        // Create a priority queue (min-heap) to store nodes
        PriorityQueue<ListNode> pq = new PriorityQueue<>(
            (a, b) -> a.val - b.val
        );

        // Push the head of each non-null list into the heap
        for (ListNode node : lists) {
            if (node != null) pq.add(node);
        }

        // Create a dummy node to build the result list
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;

        // While the heap is not empty
        while (!pq.isEmpty()) {
            // Extract the node with the smallest value
            ListNode smallest = pq.poll();

            // Add it to the result list
            tail.next = smallest;
            tail = tail.next;

            // If there's a next node, push it into the heap
            if (smallest.next != null) {
                pq.add(smallest.next);
            }
        }

        // Return the head of the merged list
        return dummy.next;
    }
}

// Driver class
public class Main {
    public static void main(String[] args) {
        Solution sol = new Solution();

        // Creating three linked lists:
        // list1: 1 -> 4 -> 5
        // list2: 1 -> 3 -> 4
        // list3: 2 -> 6

        ListNode list1 = new ListNode(1);
```

```

list1.next = new ListNode(4);
list1.next.next = new ListNode(5);

ListNode list2 = new ListNode(1);
list2.next = new ListNode(3);
list2.next.next = new ListNode(4);

ListNode list3 = new ListNode(2);
list3.next = new ListNode(6);

ListNode[] lists = new ListNode[]{list1, list2, list3};

ListNode result = sol.mergeKLists(lists);

// Print the merged list
while (result != null) {
    System.out.print(result.val + " ");
    result = result.next;
}
}

```

Complexity Analysis

Time Complexity: $O(N * \log K)$, Where N is the total number of nodes across all K linked lists. Each insertion and extraction operation on the min-heap takes $O(\log K)$ time, and we perform such operations N times (once per node). Hence, the overall time complexity becomes $O(N * \log K)$.

Space Complexity: $O(K)$, The heap stores at most K nodes at any time one from each of the K linked lists. Therefore, the auxiliary space used by the priority queue is $O(K)$. The final merged linked list uses $O(1)$ extra space if we disregard the output list, as the nodes are rearranged rather than copied.